Program 3: Working with Selenium: Maven Project To Connect To Web Servers Using Selenium Dependency

1.Sauce.demo webpage

Step 1: create a maven project on terminal by typing the command **mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false**
Step 2: Before we connect to web servers we need to have chrome driver installed. To

```
vboxuser@niranjan: ~/devop
vboxuser@niranjan:~/devop$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/vboxuser/devop
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenSeleniumApp01
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/vboxuser/devop/MyMavenSeleniumApp01
[INFO] ----------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ----------------------------------------------------------------------------
[INFO] Total time:  1.563 s
[INFO] Finished at: 2026-03-24T21:17:28+05:30
[INFO] ----------------------------------------------------------------------------
vboxuser@niranjan:~/devop$
```

download chrome driver

To check the chrome version type the command **google-chrome –version** in the terminal
extract the version and store it in a variable CHROME_VERSION

extracts and transorms JSON data

LATEST_DRIVER=$(curl -s "https://googlechromelabs.github.io/chrome-for-testing/latest-patch-versions-per-build.json" | jq -r ".builds[\"$CHROME_VERSION\"].version")

do install curl using the command
sudo apt install curl
wget https://storage.googleapis.com/chrome-for-testing-public/$LATEST_DRIVER/linux64/chromedriver-linux64.zip

Extract and move it to /usr/local/bin/:
unzip chromedriver-linux64.zip
sudo mv chromedriver-linux64/chromedriver /usr/local/bin/chromedriver
sudo chmod +x /usr/local/bin/chromedriver
Check if ChromeDriver is installed correctly:

```
vboxuser@niranjan: ~/devops
vboxuser@niranjan:~/devops$ chromedriver --version
ChromeDriver 145.0.7632.117 (de4e5d6e13b995f5e04137970c0e8ec456017467-refs/branch-heads/7632@{#3101})
vboxuser@niranjan:~/devops$
```

Step 3: edit the pom.xml file and add the dependencies

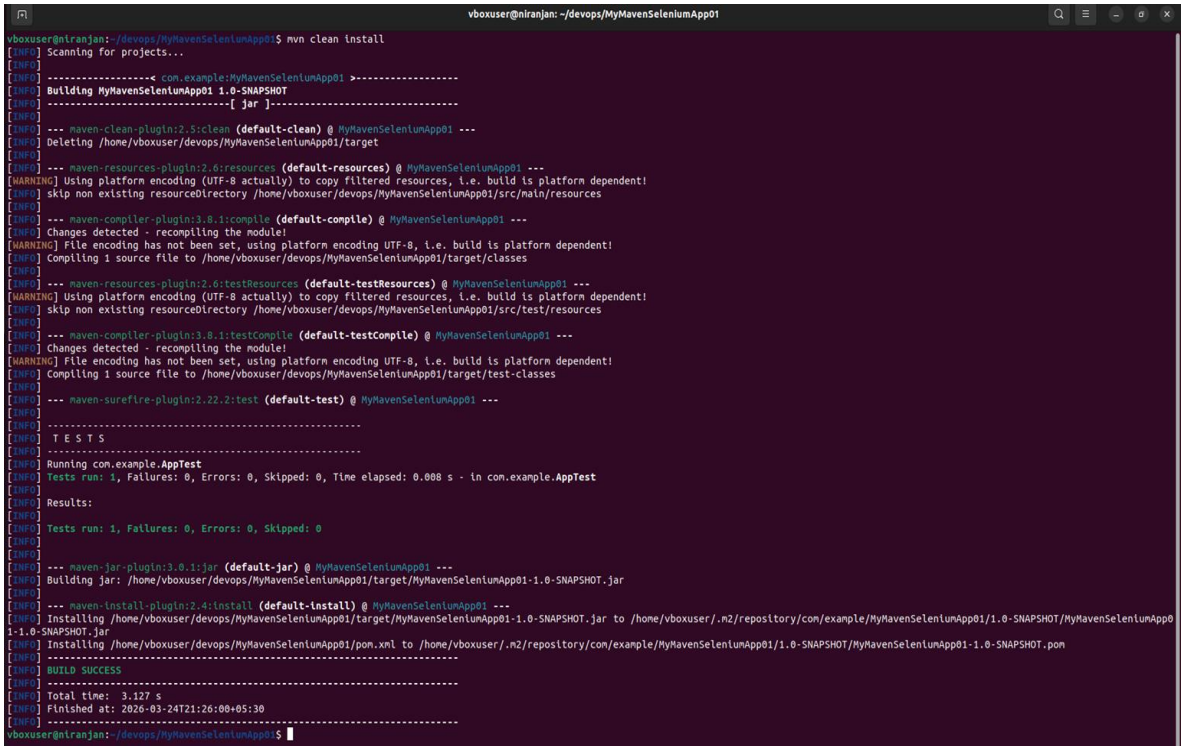
The pom.xml has the code

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.example</groupId>
 <artifactId>MyMavenSeleniumApp01</artifactId>
 <packaging>jar</packaging>
 <version>1.0-SNAPSHOT</version>
 <name>MyMavenSeleniumApp01</name>
 <url>http://maven.apache.org</url>
 <dependencies>
   <dependency>
     <groupId>junit</groupId>
     <artifactId>junit</artifactId>
     <version>3.8.1</version>
     <scope>test</scope>
   </dependency>
<dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.29.0</version>
</dependency>
  </dependencies>
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-jar-plugin</artifactId>
            <version>3.0.1</version>
           <configuration>
                <archive>
                    <manifestEntries>
                        <Main-Class>com.example.App</Main-Class>
                    </manifestEntries>
                </archive>
            </configuration>
            </plugin>
</plugins>
</build>
</project>
```

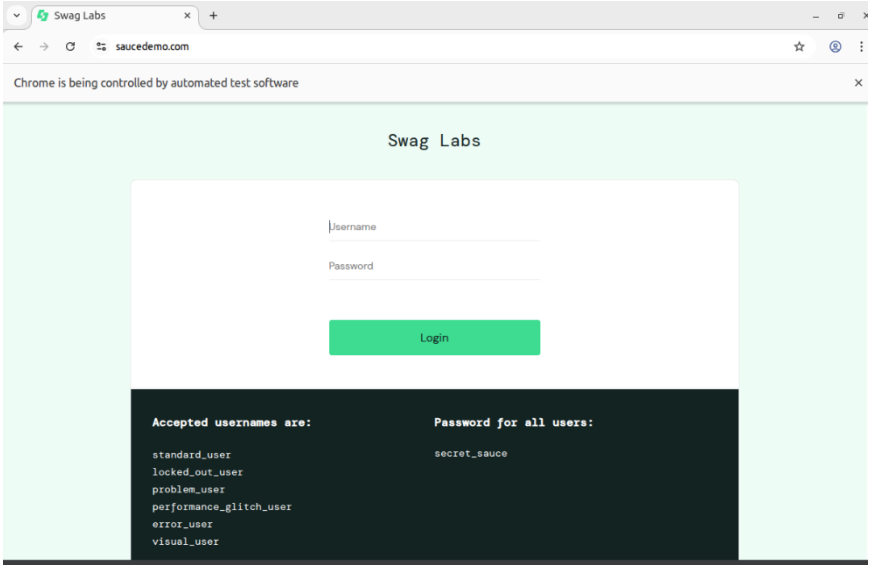
Step 4: **<u>Edit the App.java</u>**

```java
package com.example;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

/**
 * Hello world!
 *
 */
public class App
{
    public static void main( String[] args )
    {
        WebDriver driver=new ChromeDriver();
        driver.get("https://www.saucedemo.com/");
        driver.manage().window().maximize();
        driver.findElement(By.id("user-name")).sendKeys("standard_user");
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
        driver.findElement(By.id("login-button")).click();
    }
}
```

Step 5:build and run the maven application
Type **<u>mvn clean install</u>** in the terminal

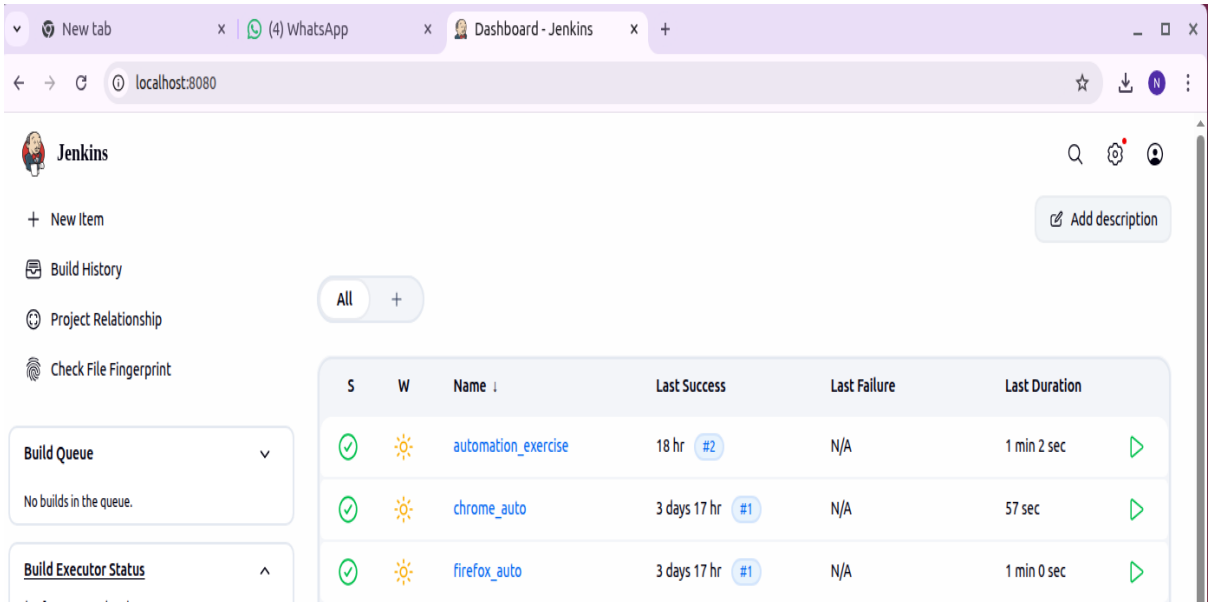


To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"** it opens **saucedemo.com** and we can login using the credentials

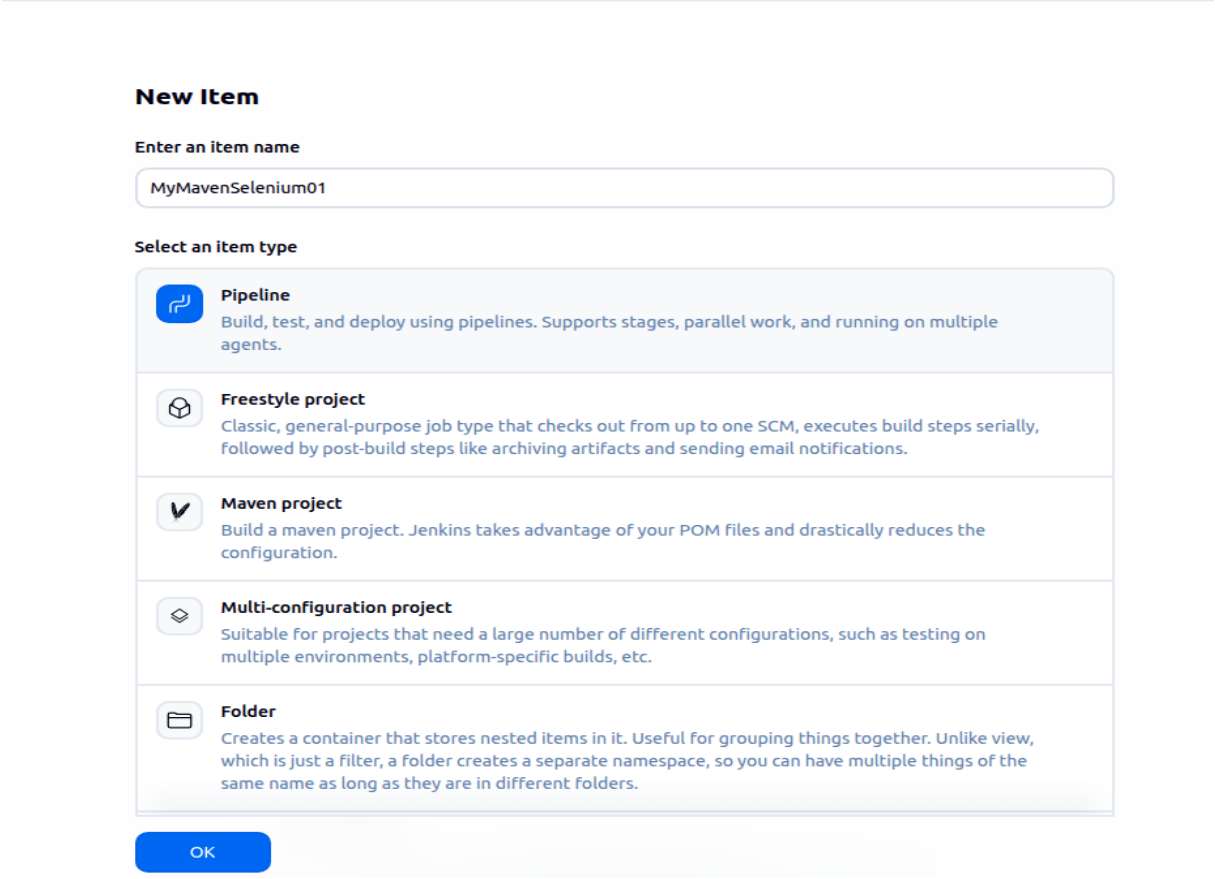


Jenkins Pipeline for MyMavenSelenium01

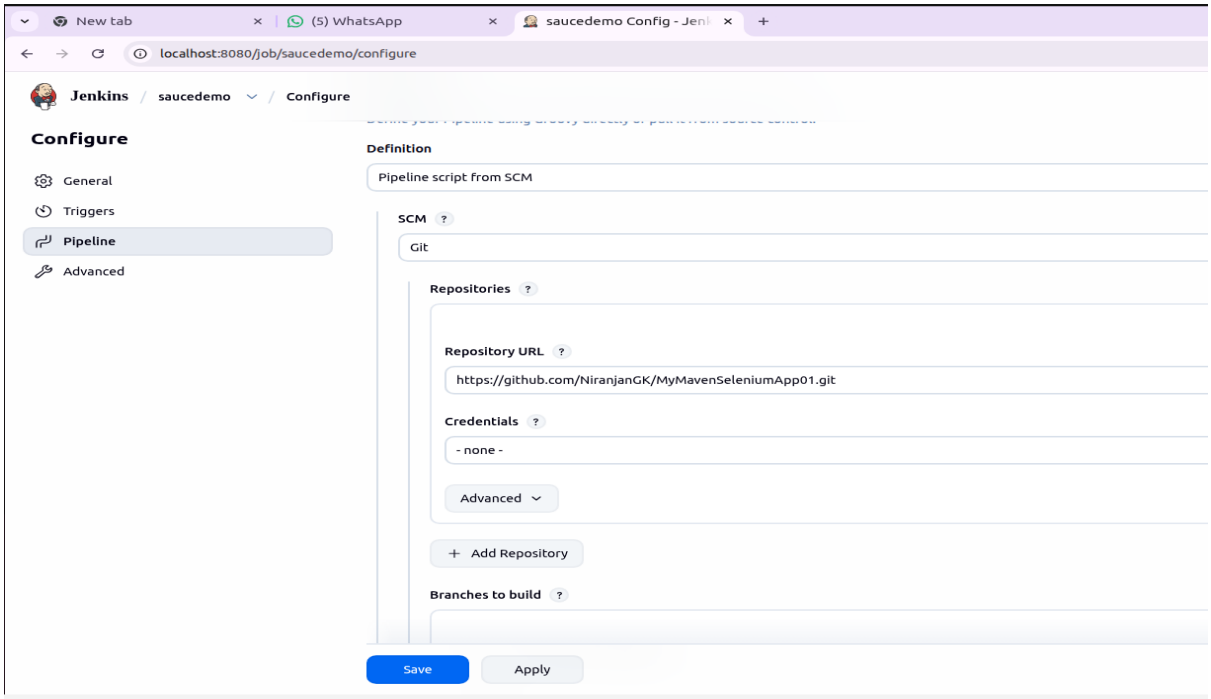
Step 1: Open the Jenkins Dashboard to create a new item



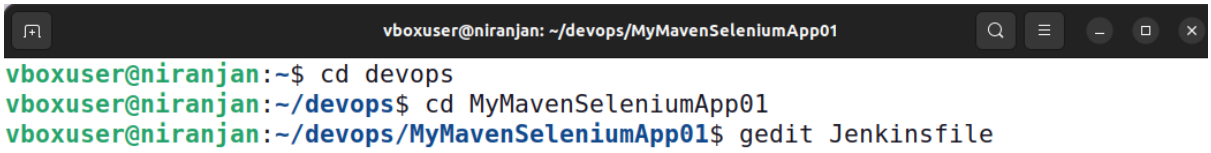
Step 2: Click on **New Item** to create New Pipeline and give the name to the pipeline and select Item type as **Pipeline** and click on **OK.**



Step 3: Configure the settings set the script as “Pipeline Script from SCM” and SCM as “Git”. Paste the Repository URL of the project given in GitHub and choose Credentials as “GitHub Credentials” which was created earlier.



Step 4: Now in Terminal, go to directory created for MyMavenAppSelenium01 and create an new file called “Jenkinsfile”



Step 5: Paste the available code inside the Jenkinsfile

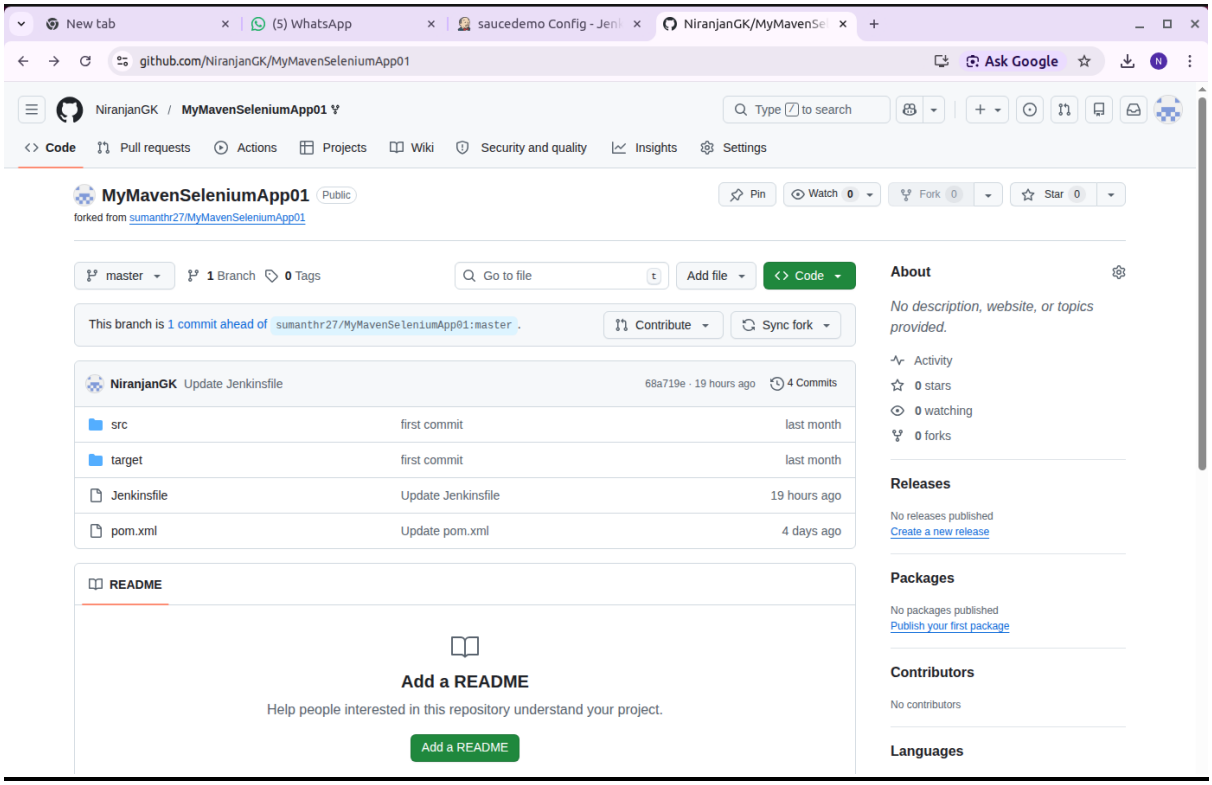
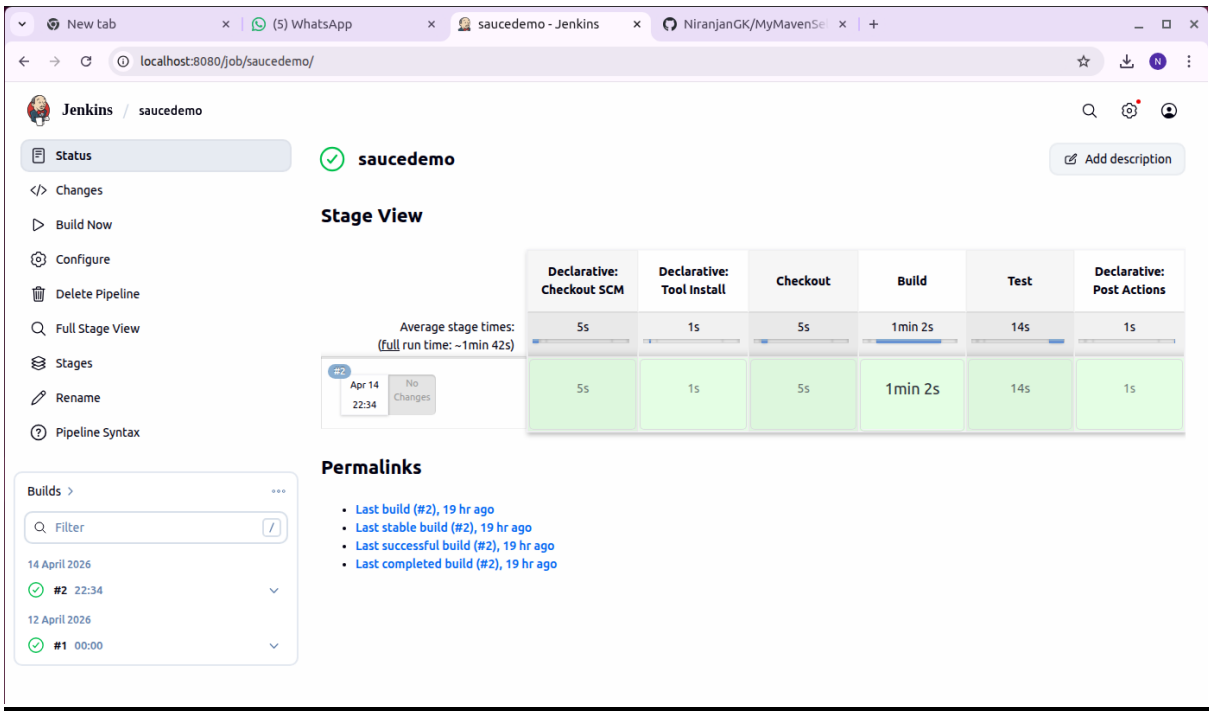
```
pipeline {
    agent any   // Run on any available agent


    tools {
        maven 'Maven'   // Must match name configured in Jenkins (Global Tool
Config)
    }


    stages {


        stage('Checkout') {
            steps {
                // Recommended if repo is configured in Jenkins
                checkout scm


                // OR use this if not configured:
                // git url:
'https://github.com/NiranjanGK/MyMavenSeleniumApp01.git', branch: 'master'
            }
        }


        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
```

```
        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }
    }

    post {
        always {
            echo 'Pipeline execution completed'
        }
        success {
            echo 'Build SUCCESS '
        }
        failure {
            echo 'Build FAILED '
        }
    }
}
```

Step 6: Make sure that the versions of source and target are correct

```
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.8</source>
<target>1.8</target>
</configuration>
</plugin>
```

Step 7: Push the Jenkins file to the Github repo



Step 8: Now go to Jenkins and Build the Pipeline and check the Stage view for the successful execution.